Actividad 02: De JSON a MySQL

Implementación y verificación del catálogo de apicultores

Rodrigo Ledezma Sanchez

Grupo HTML

Universidad Privada Domingo Savio

Ingeniería de Sistemas

Tecnología Web II

M.Sc. Ing. Jimmy Nataniel Requena Llorentty

16 de octubre de 2026

Resumen

Se completó una aplicación PHP que migra el catálogo de Apicultores Flor de la Chiquitanía a MySQL. La entrega final incorpora categorías normalizadas, relaciones con claves foráneas y CRUD de productos y socios mediante repositorios PDO. La configuración privada utiliza secretos aleatorios locales, sin contraseñas operativas en el código publicado de esta aplicación. Se implementaron validación de entrada, escape HTML, protección CSRF, paginación numerada y una alternativa por clave ID.

La verificación automatizada obtuvo 48 de 48 comprobaciones sobre PHP 8.5.8 y MySQL 8.4.11. Las pruebas abarcan operaciones por repositorio y HTTP, integridad, importación JSON, SQLi y XSS. Las capturas provienen de navegación real del asistente en el entorno local. Los planes EXPLAIN muestran menor lectura de filas en dos casos concretos, aunque las medianas locales con 36 productos no demuestran una reducción de latencia. Se distinguen estos resultados de una certificación de seguridad o rendimiento productivo.

La entrega incluye código, scripts, evidencias, bitácora consolidada de IA y presentación. La defensa oral y el envío al aula virtual permanecen a cargo del participante. La versión final amplía los ejercicios originales sin sobrescribirlos.

Palabras clave: PHP, PDO, MySQL, repositorios, SQLi, XSS.

Repositorio y versión verificable

https://github.com/ROXIAN120/Actividad-1-tecnologia-web-2/tree/main/actividad-02

Revisión del código y evidencias: ccba795bef45dcad4471650049a2c9aab0a84822. Los documentos finales se agregan después de esta revisión para evitar una referencia circular al propio PDF.

Objetivos, alcance y metodología

El objetivo general fue completar la Actividad 02 conforme a la consigna y rúbrica proporcionadas por el usuario (Universidad Privada Domingo Savio, 2026): modelo relacional, conexión PDO, CRUD de dos entidades, validación, paginación y evidencia de resistencia a inyección SQL. Para los niveles superiores se añadieron mediciones EXPLAIN, paginación por clave y pruebas XSS.

Los objetivos específicos fueron separar categorías de los productos; proteger integridad y valores monetarios; centralizar SQL preparado; ofrecer formularios con errores comprensibles; importar el catálogo JSON sin duplicar códigos; y conservar resultados reproducibles junto al código.

Se contrastaron las guías y clases disponibles con los laboratorios 8 y 9. La revisión anterior detectó CRUD de socios incompleto, categoría ENUM y ejemplos de claves publicados. La guía distingue los ejercicios del proyecto final (Requena Llorentty, 2026, pp. 31-32). La solución actual corrige estos pendientes en una aplicación independiente.

El método incluyó inspección de código y SQL, ejecución contra una base nueva, pruebas con registros temporales, navegación de formularios, captura de pantallas y análisis de planes. Se preservaron los laboratorios originales y sus evidencias históricas. La base final conserva 36 productos, ocho socios y tres categorías.

La consigna del aula indica cierre el 13 de octubre, mientras la guía local menciona otra fecha. Este informe usa la fecha real de elaboración, 16 de octubre de 2026, sin modificar el vencimiento ni afirmar recepción en la plataforma.

Matriz de trazabilidad de la entrega

Tabla 1

Requisitos y evidencia de cumplimiento técnico

Requisito	Implementación final	Evidencia
Modelo y SQL	Tres tablas, PK, FK, UNIQUE, CHECK e índices	sql/01-esquema.sql; 02-datos.sql; modelo ER
Normalización e índices	Dependencias separadas y planes medidos	Diseño del informe; EXPLAIN_MEDICIONES.json
PDO privado	Configuración fuera de public, preparación nativa	config/bd.php; Conexion.php; .env ignorado
CRUD de productos	Crear, leer, editar y borrar	ProductoRepositorio; pruebas y figuras
CRUD de socios	Crear, leer, editar y borrar con restricción FK	SocioRepositorio; pruebas y figuras
Paginación	Numerada en ambos; por clave ID en productos	Pagina; buscar(); despues()
Validación y errores	422, 409, 404 y 500 privado	Validacion; ErrorHttp; front controller
SQLi y XSS	Marcadores, lista permitida y escape	Casos automatizados y capturas
Informe y bitácora	APA 7, código explicado e historial de IA	Este informe; BITACORA_IA.md
Presentación y defensa	Diapositivas y guion de demostración	Presentación final; GUIA_DEFENSA.md

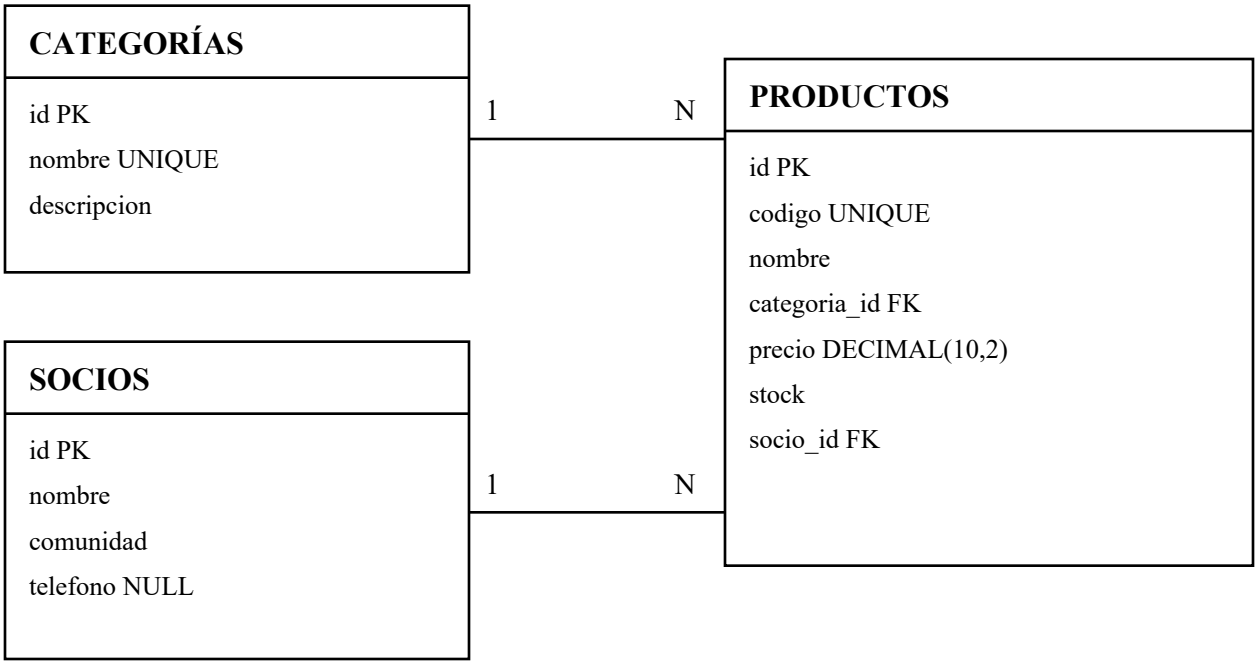
La matriz acredita la implementación y los casos comprobados, no una calificación automática. La defensa humana y la entrega en el aula no se consideran realizadas. El formato del informe utiliza papel carta, márgenes de 2,54 cm, Times New Roman 12, doble espacio en el cuerpo y referencias con sangría francesa. Tablas, código y notas usan tamaños y espaciado adaptados a su legibilidad.

Modelo entidad-relación y normalización

Cada producto pertenece a una categoría y a un socio. Una categoría o un socio pueden tener cero o varios productos. Las FK no nulas impiden productos sin clasificación o productor; ON DELETE RESTRICT evita dejar referencias huérfanas al borrar socios o categorías.

Figura 1

Modelo relacional implementado



Nota. Modelo de la base actividad02. El archivo sql/MODELO_ER.mmd conserva su descripción editable. PK: clave primaria. FK: clave foránea.

La primera forma normal se satisface con atributos de un solo valor y filas identificadas. Las claves primarias simples evitan dependencias parciales. La tercera forma normal se apoya en separar nombre y descripción de categoría, y nombre, comunidad y teléfono del socio: estos datos no se repiten como atributos del producto. El precio depende del producto, no del socio. Esta decisión evita actualizar muchos productos cuando cambia una comunidad o el nombre de una categoría.

Diccionario de datos y restricciones

Tabla 2

Atributos principales de la base

Tabla	Campo y tipo	Regla
categorias	id INT UNSIGNED; nombre VARCHAR(60); descripcion VARCHAR(150)	PK; nombre único; campos no nulos
socios	id INT UNSIGNED; nombre VARCHAR(80); comunidad VARCHAR(60)	PK; nombre 3-80 y comunidad 2-60 mediante CHECK
socios	telefono VARCHAR(20)	NULL permitido; formato validado en PHP
productos	id INT UNSIGNED; codigo VARCHAR(8); nombre VARCHAR(80)	PK; código UNIQUE; nombre 3-80
productos	categoria_id y socio_id INT UNSIGNED	FK no nulas; borrado RESTRICT
productos	precio DECIMAL(10,2)	CHECK: mayor que cero y hasta 10000
productos	stock INT	CHECK: entero entre 0 y 100000

Se eligió InnoDB, utf8mb4 y la colación utf8mb4_0900_ai_ci. Los identificadores son compatibles entre PK y FK. El código comercial es único y permanece inmutable en la edición. DECIMAL representa importes de dos decimales; PHP normaliza la entrada a una cadena decimal, sin usar un cálculo monetario con float como dato persistido.

Los índices (nombre,id), (categoria_id,nombre,id) y (socio_id,nombre,id) apoyan orden estable y búsquedas por relación. El índice del nombre de socio acompaña su listado. No se afirma que LIKE con comodín inicial aproveche una búsqueda por prefijo; ese filtro puede exigir recorrer registros. Los planes concretos se presentan más adelante.

Migración del catálogo JSON a MySQL

El archivo migracion/origen_actividad01.json conserva los seis productos del catálogo anterior. El esquema y la semilla SQL crean una base reproducible con 36 productos, ocho socios y tres categorías. Esta inicialización amplía el catálogo original y mantiene sus códigos y valores.

ImportadorJson valida estructura y contenido, consulta categorías y socios, y usa una transacción. El modo predeterminado revierte cambios para simular. Con --aplicar confirma la transacción. Si el código existe, lo omite sin sobrescribirlo; si falta comunidad para un socio nuevo, registra “Sin especificar”, porque el JSON original no contiene ese dato.

La ejecución documentada sobre la semilla leyó seis productos, omitió los seis existentes y no creó nuevos. No sería correcto presentar esa ejecución como seis inserciones nuevas. Las pruebas automatizadas sí verificaron un producto temporal con simulación revertida, inserción confirmada y segunda ejecución sin duplicados.

Comandos de importación

```
php importar_json.php
php importar_json.php --aplicar
php importar_json.php "ruta/al/catalogo.json" --aplicar
```

Decisiones de consistencia

La transacción evita una importación parcial ante JSON inválido o una relación incorrecta. La validación comparte las reglas del formulario y los repositorios mantienen SQL preparado. El script inicial no contiene DROP y la nueva instancia no reutiliza los volúmenes de las tareas 8 y 9. No se migraron automáticamente modificaciones posteriores que el usuario pudiera haber agregado a otras bases.

Arquitectura y recorrido de una solicitud

La aplicación conserva separación entre acceso a datos y presentación. public/index.php recibe la solicitud y determina ruta y método. Validacion comprueba entradas; los repositorios ejecutan SQL con PDO; las plantillas convierten el resultado en HTML escapado. Composer resuelve las clases del espacio App.

En un alta, el navegador envía POST con token CSRF. El controlador valida el token, obtiene campos escalares, comprueba reglas y relaciones, y solicita crear() al repositorio. Si la escritura tiene éxito, responde 303 para redirigir al listado y mostrar un mensaje. Este patrón evita repetir el POST al refrescar la página.

Componente	Responsabilidad concreta
public/index.php	Rutas, métodos HTTP, sesión, CSRF, PRG y captura de errores
src/Conexion.php y config/bd.php	PDO y lectura obligatoria de configuración privada
src/Repositorio.php	prepare, enlace tipado, execute y traducción de conflictos
ProductoRepositorio / SocioRepositorio	SQL específico, CRUD y consultas paginadas
CategoriaRepositorio / Validacion	Categorías válidas, reglas y existencia de relaciones
plantillas/*.php	Formularios, listados, fichas y errores con escape
ImportadorJson / pruebas/	Importación transaccional y verificación reproducible

Los repositorios actuales son clases concretas que heredan un ejecutor común. No se afirma que exista un contrato de intercambio con memoria en esta aplicación final: ese ejercicio pertenece al laboratorio 9 histórico. El diseño final privilegia trazabilidad y CRUD completo en MySQL.

Explicación de código: conexión y ejecución preparada

```
// src/Conexion.php: opciones reales de PDO
PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
PDO::ATTR_EMULATE_PREPARES => false,
PDO::MYSQL_ATTR_MULTI_STATEMENTS => false,

// src/Repositorio.php: núcleo de consultar()
$c = $this->bd->prepare($sql);
foreach ($valores as $clave => $valor) {
    $c->bindValue(is_int($clave) ? $clave + 1 : $clave,
        $valor, is_int($valor) ? PDO::PARAM_INT :
        ($valor === null ? PDO::PARAM_NULL : PDO::PARAM_STR));
}
$c->execute();
return $c;
```

Conexion carga config/bd.php fuera del directorio público y construye un DSN utf8mb4. Las claves provienen del entorno o del archivo .env local ignorado. configurar.php genera valores aleatorios una sola vez, sin imprimirlos. El archivo de ejemplo contiene marcadores y no sirve como contraseña operativa.

prepare separa la estructura SQL de los valores. bindValue enlaza enteros, nulos o cadenas según su tipo; el desplazamiento +1 adapta el arreglo PHP con índices desde cero a los marcadores posicionales de PDO desde uno. execute envía la operación y consultar devuelve el statement para fetch, fetchAll o fetchColumn. Las consultas estáticas también siguen este camino.

PHP explica que los parámetros protegen los valores, pero no otras partes de SQL construidas desde texto externo sin control (PHP Documentation Group, s. f.). Por eso la aplicación no concatena nombres de columnas recibidos del usuario: ORDER BY se elige de constantes permitidas.

Explicación de código: CRUD de productos

```
// Fragmentos de src/ProductoRepositorio.php
public function crear(array $d): void {
    $this->escribir('INSERT INTO productos
      (codigo,nombre,categoria_id,precio,stock,socio_id)
      VALUES (:codigo,:nombre,:categoria_id,:precio,:stock,:socio_id)', $d);
}
// Lectura por código: consulta base con JOIN y WHERE p.codigo=?
// Edición: UPDATE ... SET nombre=:nombre, ... WHERE codigo=:codigo
// Borrado: DELETE FROM productos WHERE codigo=?
```

crear recibe exclusivamente los campos ya validados. porCodigo consulta el producto con JOIN de categorías y socios para mostrar nombres coherentes. actualizar comprueba existencia antes de modificar y conserva el código de la ruta. eliminar traduce cero filas afectadas a 404. Las formas de edición y borrado usan POST con CSRF, no enlaces GET destructivos.

El método buscar aplica el mismo WHERE a COUNT y al listado, de modo que el total corresponde al filtro. La consulta de filas incluye JOIN y evita una consulta adicional por producto. La lista permitida de orden contiene nombre, precio, precio descendente y stock; cada orden incorpora ID como desempate estable.

Un código duplicado provoca el error MySQL 1062, que el repositorio convierte a conflicto 409. Una FK inválida o un CHECK incumplido también se maneja como conflicto. Los errores inesperados no muestran consultas ni contraseñas al visitante. Los fragmentos comentados anteriores resumen métodos completos disponibles en el repositorio, sin sustituir el código entregado.

Explicación de código: CRUD y ficha de socios

```
// Fragmentos de src/SocioRepositorio.php
public function crear(array $d): int {
    $this->escribir('INSERT INTO socios (nombre,comunidad,telefono)
      VALUES (:nombre,:comunidad,:telefono)', $d);
    return (int)$this->bd->lastInsertId();
}
// Actualización parametrizada
UPDATE socios SET nombre=:nombre,comunidad=:comunidad,
    telefono=:telefono WHERE id=:id
// Eliminación parametrizada
DELETE FROM socios WHERE id=?
```

SocioRepositorio incorpora todos, porId, buscar, crear, actualizar y eliminar. crear devuelve el ID asignado por MySQL. El teléfono vacío se normaliza a NULL. actualizar comprueba que exista el socio; eliminar comunica 404 si ya no existe y 409 si una FK impide quitarlo por sus productos asociados.

buscar pagina diez socios, filtra por nombre o comunidad y cuenta productos con LEFT JOIN y GROUP BY. El total de socios se calcula por separado, sin multiplicarlo por sus productos. Con ocho socios iniciales solo hay una página; las pruebas crean registros temporales para comprobar una segunda página real y después los eliminan selectivamente.

La ficha /socios/{id} muestra datos del productor y productos relacionados. La regla de negocio exige reasignar o eliminar primero los productos antes de borrar su socio. Se conserva la integridad referencial, en lugar de eliminar silenciosamente todo el inventario de ese productor.

Validación, errores y consistencia de la interfaz

Entrada o evento	Comportamiento
Código de producto	Formato AAA-[A-Z0-9] de 2 a 4 caracteres finales; máximo 8
Nombre	Entre 3 y 80 caracteres UTF-8
Precio	0,01 a 10000,00; hasta dos decimales; coma normalizada
Stock	Entero entre 0 y 100000
Socio y categoría	IDs válidos y relaciones existentes
Comunidad / teléfono	Comunidad 2-60; teléfono opcional con formato restringido
Formulario inválido	HTTP 422, errores por campo y valores conservados escapados
Duplicado, FK o CHECK	HTTP 409 con mensaje comprensible
Token o método incorrecto	HTTP 403 o 405, sin escritura
Error inesperado	HTTP 500 con referencia; detalle en log privado

Validacion rechaza arreglos donde se espera texto, verifica UTF-8 y normaliza importes sin convertir el dato persistente en float. La base repite reglas esenciales con CHECK, UNIQUE y FK: el formulario orienta al usuario, mientras SQL conserva integridad incluso ante otros clientes o cambios concurrentes.

La interfaz conserva búsqueda, categoría y orden al navegar. La ordenación por clave se identifica como ID ascendente, sin prometer orden por nombre ni conteos. El stock cero se admite y se muestra como valor real. No se afirma disponibilidad a partir de datos inventados o de registros ya eliminados.

Paginación y conteos eficientes

```
// Numerada: total y página limitada con parámetros enteros
SELECT COUNT(*) FROM productos p WHERE ...
SELECT ... FROM productos p JOIN ... WHERE ...
ORDER BY p.nombre, p.id LIMIT :lim OFFSET :off

// Por clave: después del ID recibido y sin consulta COUNT
SELECT ... FROM productos p JOIN ... WHERE ...
AND p.id > :id ORDER BY p.id LIMIT :lim
// Se piden 11 filas: se muestran 10 y la extra indica continuación.
```

La página numerada limita a diez filas y ajusta números fuera de rango. Con 36 productos hay cuatro páginas y seis filas en la última. Los conteos no arrastran los JOIN de presentación. Las entradas no numéricas vuelven a la primera página; filtros y enlaces mantienen sus valores mediante codificación de URL.

La alternativa por clave usa el último ID visible como cursor. Evita descartar filas con OFFSET y elimina COUNT del recorrido. Se pide una fila adicional para decidir si existe un siguiente bloque. Conserva búsqueda y categoría, pero fija el orden por ID y no permite saltos directos a una página arbitraria.

Esta alternativa se implementa en productos. Los socios mantienen paginación numerada y conteo independiente. No se presenta el cursor como equivalente a un cursor compuesto por nombre o precio, ni se asegura una ventaja para toda carga posible. Las mediciones siguientes comparan dos consultas concretas sobre los datos iniciales.

Índices y mediciones EXPLAIN

Tabla 3

Planes y medianas de 31 ejecuciones por consulta

Consulta	Acceso observado	Filas leídas[1]	Mediana ms[2]
Socio sin índice compuesto	idx_producto_nombre	36	0.6655
Socio con índice compuesto	idx_producto_socio_nombre	4	0.7093
OFFSET 20, LIMIT 10	PRIMARY (index scan)	30	0.7631
ID > 20, LIMIT 10	PRIMARY (range scan)	10	0.8207

Nota. [1] Filas observadas en el operador de acceso de EXPLAIN ANALYZE, no estimaciones rows de EXPLAIN. [2] Tiempo de ejecución observado por PHP en el entorno local. MySQL 8.4.11, 36 productos. Fecha: 2026-10-16T19:07:20-04:00.

pruebas/medir.php actualiza estadísticas con ANALYZE TABLE y registra EXPLAIN, EXPLAIN ANALYZE y medianas. IGNORE INDEX desactiva el índice compuesto en una consulta comparativa sin cambiar el esquema. La consulta de socio con índice utiliza acceso ref y cobertura de índice para cuatro resultados.

En el ejemplo de paginación, OFFSET necesita recorrer 30 filas para devolver diez; la condición por clave devuelve diez con un recorrido de rango. Sin embargo, las medianas de ambas comparaciones son ligeramente mayores con el índice compuesto y con clave en esta ejecución. No se concluye una reducción de latencia. La muestra pequeña y la comunicación local limitan cualquier extrapolación.

Oracle (s. f.) distingue EXPLAIN, que describe el plan, de EXPLAIN ANALYZE, que ejecuta la consulta y aporta tiempos y filas reales. El archivo JSON conserva ambos resultados íntegros y las cuatro consultas para repetir el análisis.

Seguridad: SQLi, XSS y CSRF

Las entradas de búsqueda, códigos, IDs y campos de escritura se enlazan como parámetros. Los únicos fragmentos SQL variables corresponden a condiciones internas y órdenes constantes permitidos. LIKE escapa los caracteres %, _ y barra invertida para tratarlos como texto literal. Un intento SQLi no se usa para construir instrucciones ejecutables.

El caso zzz' OR 1=1 -- - devolvió cero resultados en productos y socios. La comilla sola no produjo error SQL y el porcentaje se trató como texto literal. Estas comprobaciones, junto a la inspección de todos los puntos de consulta, respaldan los controles implementados para el alcance examinado. No equivalen a un análisis universal de seguridad.

```
// public/index.php: escape para texto y atributos HTML
htmlspecialchars((string)$v, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')
// XSS de prueba: entrada literal, nunca código de la página
<script>alert(1)</script>
```

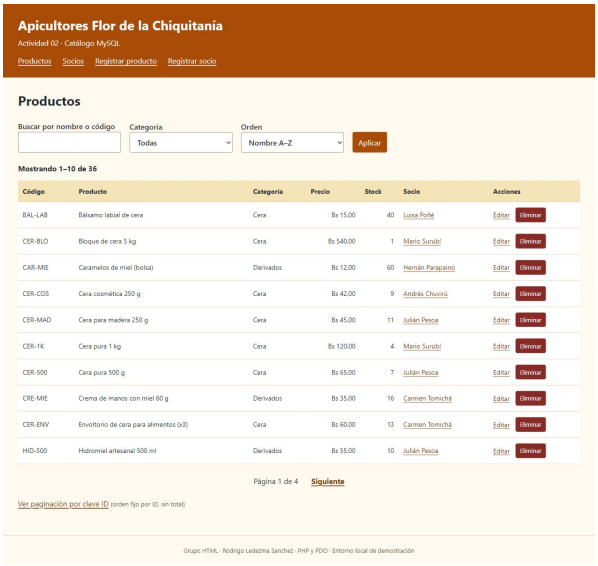
La salida dinámica pasa por escape HTML, incluidos mensajes, formularios, nombres y búsquedas. Las pruebas verifican XSS reflejado y almacenado como texto escapado. La CSP bloquea scripts y objetos, como defensa adicional. OWASP Foundation (s. f.) recomienda codificar según el contexto de salida; aquí la aplicación no inserta valores en JavaScript ni permite HTML enriquecido.

Las escrituras exigen POST y token de sesión comparado de forma segura. Las cookies usan HttpOnly y SameSite=Lax, con Secure si existe HTTPS. Las respuestas añaden nosniff y política de referencia. El documento público no expone .env ni config/bd.php. Los logs quedan fuera de public y de Git. La aplicación es local y carece de autenticación: no debe desplegarse públicamente como sistema productivo.

Evidencias: catálogo y filtro

Figura 2

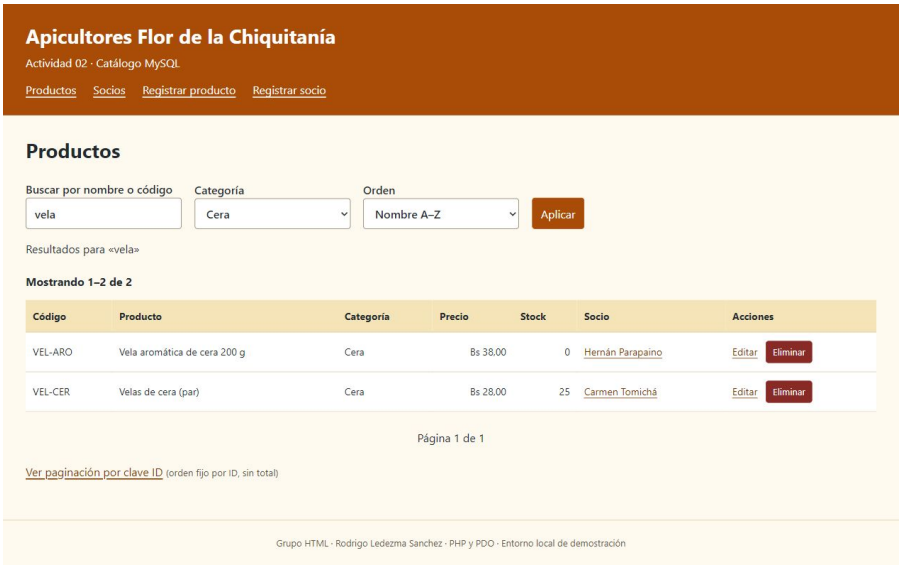
Catálogo final con paginación



Nota. Navegación real del asistente, 16 de octubre de 2026, /productos. Base de 36 productos.

Figura 3

Categoría Cera y búsqueda vela

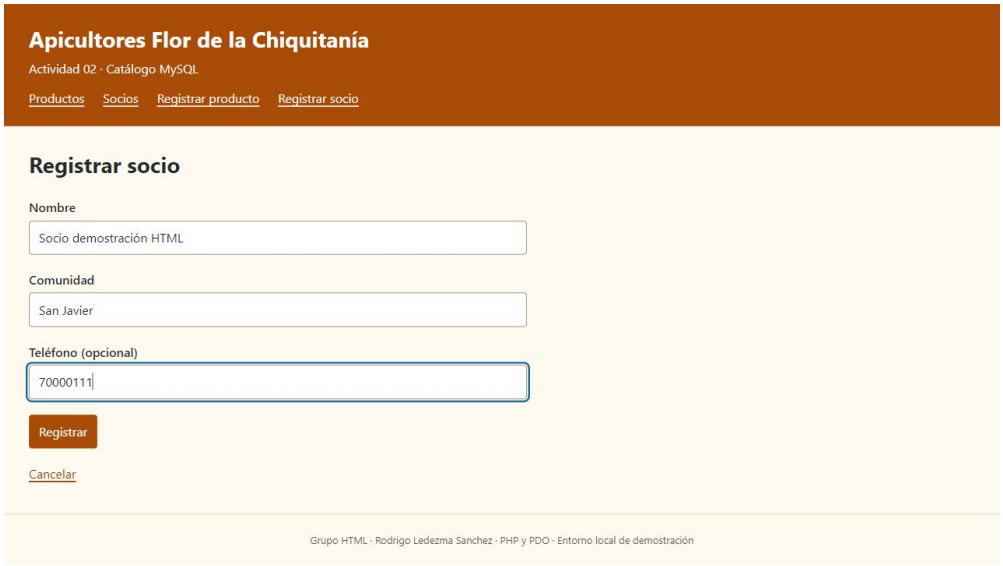


Nota. /productos?categoria=3&q=vela. Dos coincidencias coherentes con los datos iniciales.

Evidencias: creación y edición de socios

Figura 4

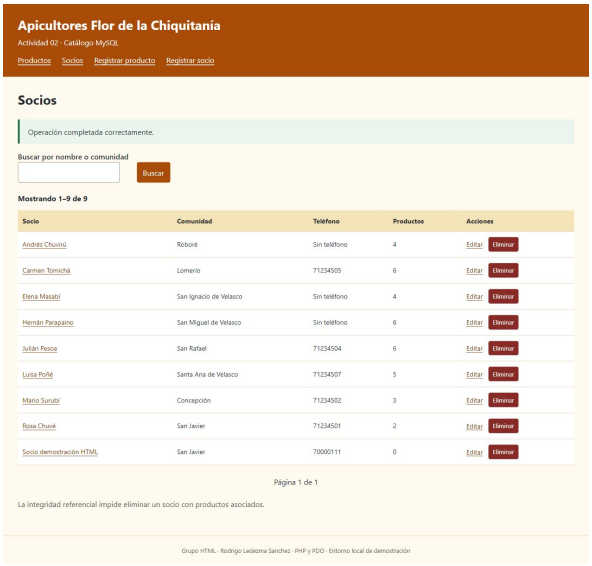
Formulario de socio de demostración



Nota. Datos ficticios de prueba. El teléfono de demostración no se presenta como contacto real.

Figura 5

Socio creado correctamente

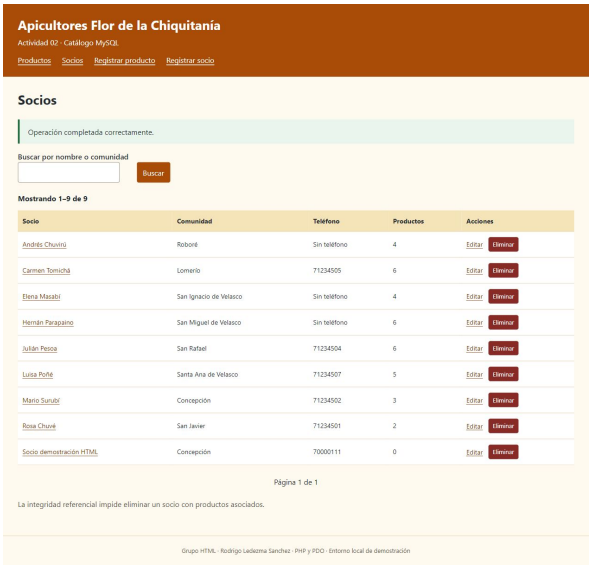


Nota. El listado aumenta temporalmente a nueve socios. La prueba manual ejercita POST y redirección.

Evidencias: cierre del CRUD de socios

Figura 6

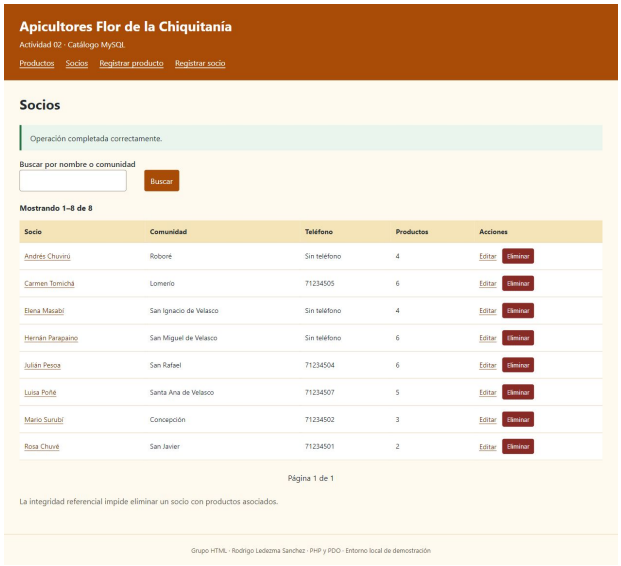
Edición de comunidad del socio temporal



Nota. La comunidad cambia a Concepción. El registro es exclusivamente de demostración.

Figura 7

Eliminación del socio temporal

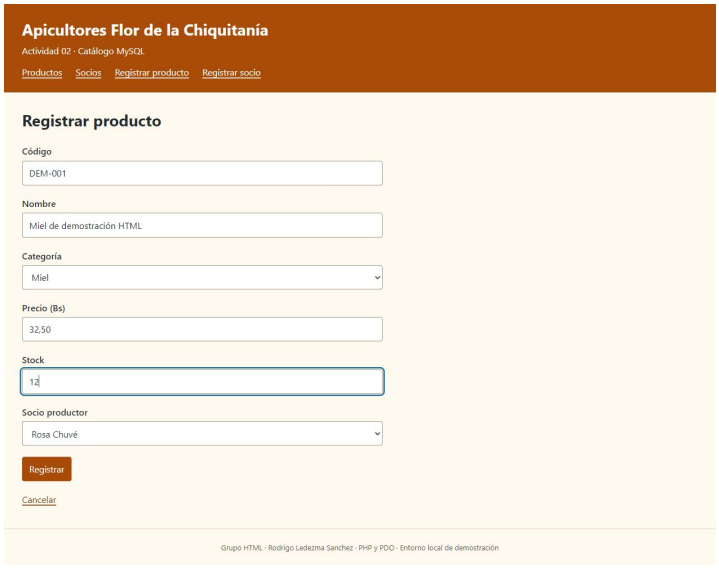


Nota. El listado vuelve a ocho socios. Los datos iniciales permanecen y no se restauran tablas.

Evidencias: creación de producto

Figura 8

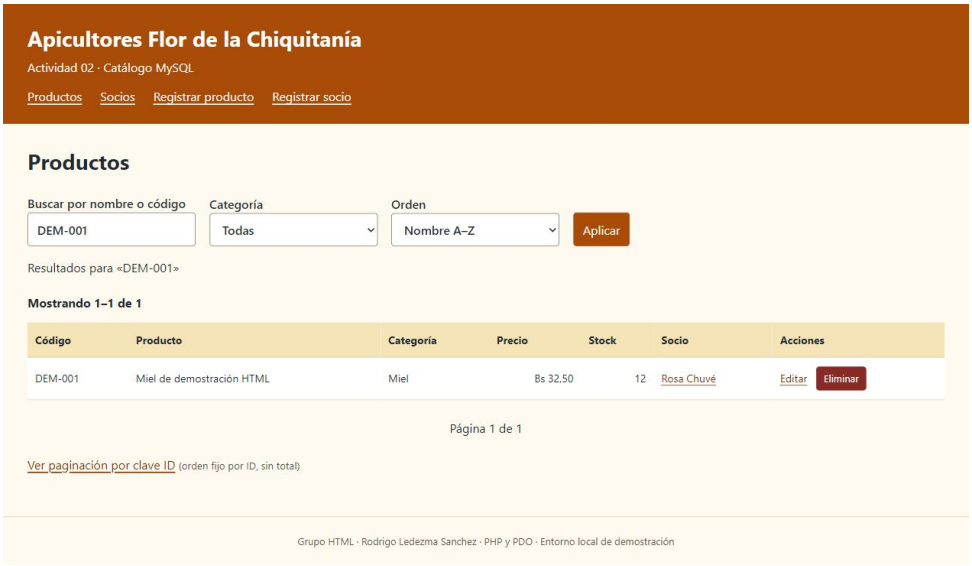
Formulario de producto DEM-001



Nota. Precio de prueba 32,50, categoría Miel y socio válido. Los datos se validan antes de persistir.

Figura 9

Producto de demostración creado

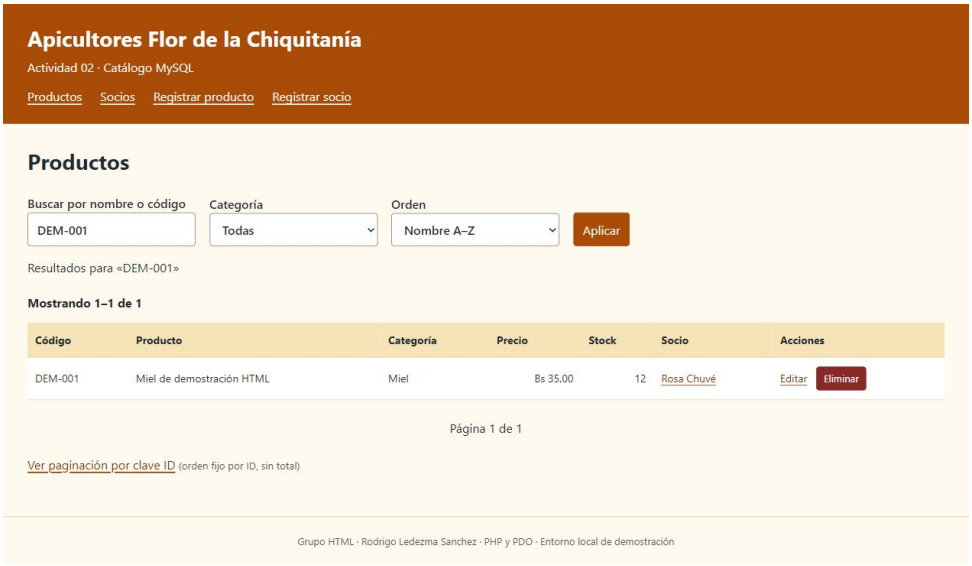


Nota. La búsqueda por DEM-001 muestra una coincidencia con Bs 32,50.

Evidencias: edición y eliminación de producto

Figura 10

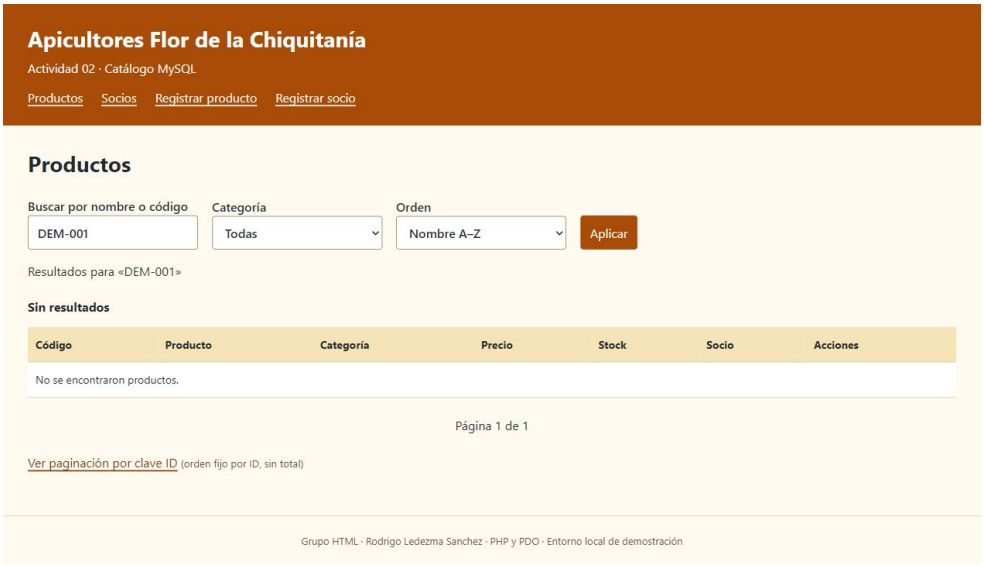
Precio actualizado del producto temporal



Nota. La misma clave comercial DEM-001 muestra Bs 35,00 tras la edición.

Figura 11

Producto temporal eliminado

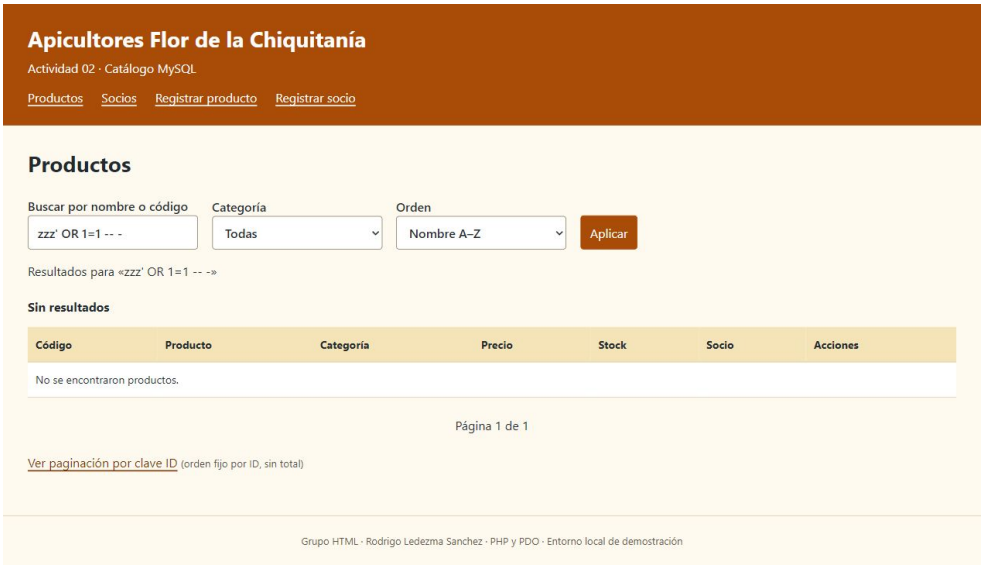


Nota. La búsqueda por DEM-001 vuelve a cero coincidencias. La base final conserva los 36 productos originales.

Evidencias: resistencia a SQLi y XSS

Figura 12

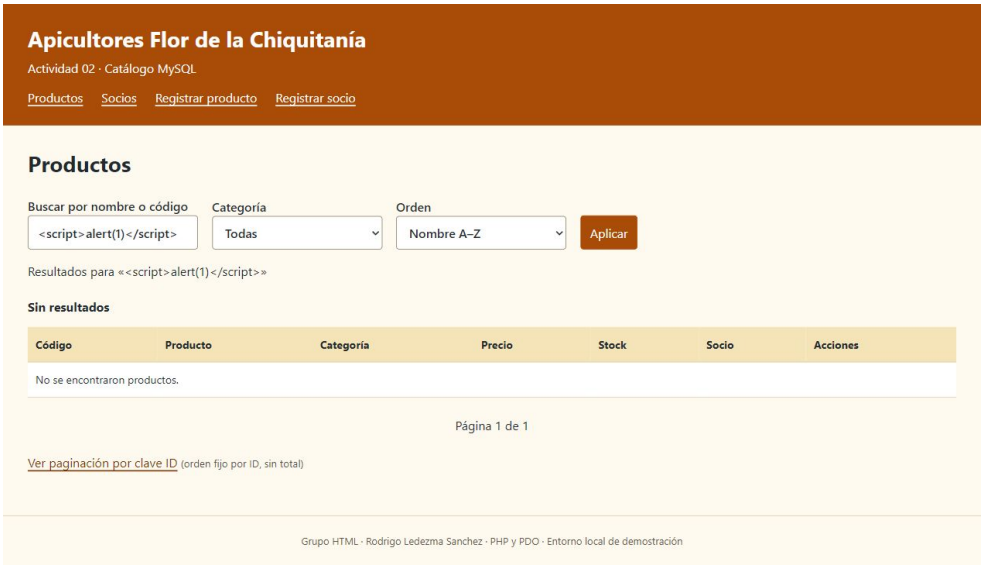
Intento de inyección SQL sin coincidencias



Nota. Cadena zzz' OR 1=1 -- -. La aplicación no devuelve todo el inventario ni muestra error SQL.

Figura 13

Entrada XSS mostrada como texto

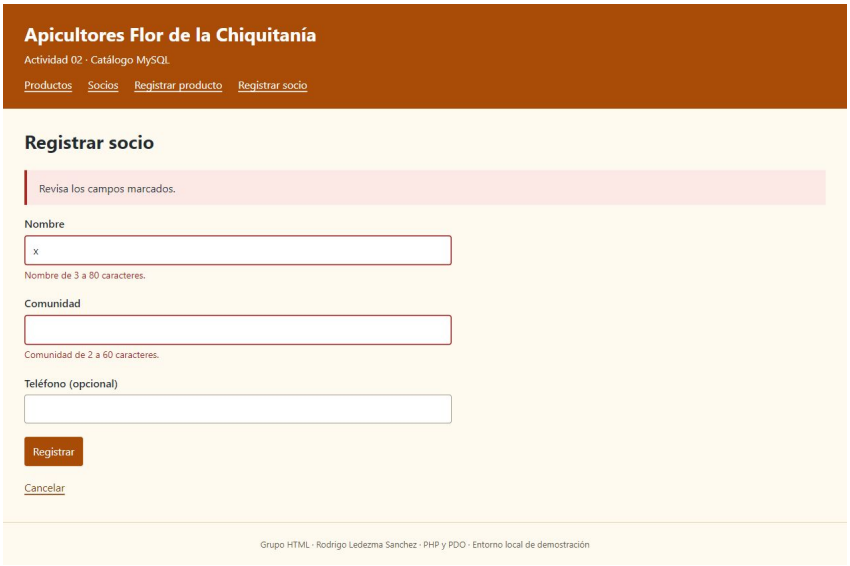


Nota. La búsqueda contiene <script>alert(1)</script> sin ejecución. La prueba almacenada se acredita además en el verificador.

Evidencias: validación y recorrido por clave

Figura 14

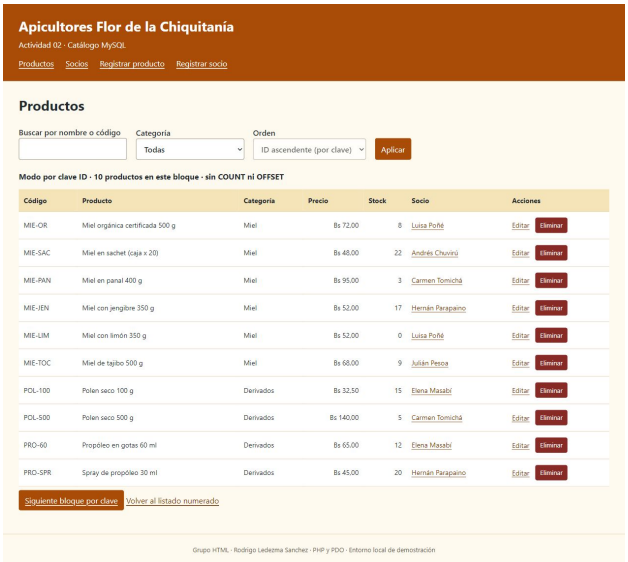
Validación del formulario de socios



Nota. Nombre de una letra y comunidad vacía provocan errores de campo. El servidor responde 422.

Figura 15

Segundo bloque por clave ID



Nota. /productos?modo=cursor&despues=10. La interfaz declara orden por ID y ausencia de COUNT y OFFSET.

Pruebas automatizadas y resultados

La ejecución final de php pruebas/verificar.php obtuvo 48 de 48 comprobaciones. El script inicia un servidor PHP de prueba, conserva cookies y token CSRF, y contrasta respuestas HTTP con consultas de repositorios. Los archivos RESULTADOS_PRUEBAS.txt y .json conservan el detalle y el entorno.

Los fixtures incluyen socios, productos y una entrada XSS almacenada. La limpieza final elimina únicamente registros creados por las pruebas. No restaura tablas ni borra datos del usuario. El verificador espera el catálogo inicial; si el usuario lo modifica después, las aserciones de conteo pueden fallar sin que eso justifique borrar su información.

Cobertura	Comprobaciones representativas
Base y listados	PDO nativo; categorías; 36/8; última página; filtros; orden
Repositorios	Crear, actualizar y eliminar ambas entidades; duplicado; FK; CHECK
Migración	Simulación revertida; inserción confirmada; repetición idempotente
HTTP	PRG 303; 422; 403; 405; 404; CRUD completo
Seguridad	SQLi en ambos listados; XSS reflejado/almacenado; archivos privados
Paginación y limpieza	Socios con segunda página; cursor sin duplicados; fixtures eliminados

Los casos no reemplazan pruebas de concurrencia, carga, auditoría independiente o autorización de usuarios. El resultado se limita al código y entorno documentados. Los 15/15 y 14/14 de los laboratorios anteriores son evidencia histórica distinta y no se suman a estas 48 pruebas de la aplicación final.

Reproducción y preparación de la entrega

En Windows, iniciar Docker Desktop y comprobar que el motor esté disponible. Clonar el repositorio y entrar a actividad-02. Se requiere PHP 8.3 o superior con pdo_mysql, mbstring y curl, además de Composer. Las versiones comprobadas fueron PHP 8.5.8 y MySQL 8.4.11.

```
git clone https://github.com/ROXIAN120/Actividad-1-tecnologia-web-2.git
cd Actividad-1-tecnologia-web-2/actividad-02
php configurar.php
docker compose up -d --wait mysql
composer dump-autoload
php pruebas/verificar.php
php pruebas/medir.php
php importar_json.php
php -S 127.0.0.1:8020 -t public public/index.php
```

Abrir http://127.0.0.1:8020/productos y /socios. La base utiliza el puerto 3309 y un volumen exclusivo. La inicialización SQL solo ocurre en un volumen nuevo. Se debe conservar el .env asociado al volumen existente: cambiar su contraseña no actualiza automáticamente los usuarios MySQL. No usar down -v para resolver problemas, porque borra datos.

El paquete final contiene aplicación, scripts, capturas, mediciones, informe, presentación y guía de defensa, sin .env ni vendor. Composer regenera el cargador. Para detener servicios sin eliminar datos, usar docker compose stop; para el servidor PHP, Ctrl+C. La entrega al aula y la defensa requieren acción humana.

Conclusiones y límites

La aplicación final completa las funciones técnicas solicitadas: esquema con categorías, socios y productos; integridad relacional; configuración privada; CRUD preparado de ambas entidades; validación; paginación y manejo de errores. Las capturas y las 48 comprobaciones aportan evidencia reproducible de esos escenarios.

La importación conserva el catálogo JSON y documenta honestamente la omisión de códigos ya presentes. El análisis de índices identifica reducción de filas leídas, pero las mediciones de una base pequeña no permiten prometer menor latencia general. La alternativa por clave tiene un orden explícito por ID y un alcance distinto de la navegación numerada.

Los controles de SQLi y XSS se implementan en capas y cuentan con casos de prueba. No se encontraron consultas que concatenen valores del usuario dentro de la aplicación final revisada. Esto no constituye una certificación universal ni convierte el sistema local en un servicio productivo. Antes de un despliegue real se necesitarían autenticación, roles, TLS, respaldos y evaluación de carga.

El repositorio conserva los laboratorios como historia formativa y agrega la solución final en actividad-02. Sus ejemplos educativos antiguos no deben confundirse con los secretos aleatorios actuales. La bitácora declara la participación sustancial de IA en programación y redacción. El responsable deberá revisar integrantes y datos de portada, comprender el código y defender los resultados.

Referencias

OpenAI. (2026). *Codex* [Asistente de inteligencia artificial]. https://openai.com/codex/

Oracle. (s. f.). *EXPLAIN statement. MySQL 8.4 reference manual.* https://dev.mysql.com/doc/refman/8.4/en/explain.html

OWASP Foundation. (s. f.). *Cross site scripting prevention cheat sheet.* https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

PHP Documentation Group. (s. f.). *Prepared statements and stored procedures. PHP manual.* https://www.php.net/manual/en/pdo.prepared-statements.php

Requena Llorentty, J. N. (2026). *Guía didáctica Docker Lab01-Lab09 (updated)* [Material de clase]. Universidad Privada Domingo Savio.

ROXIAN120. (2026). *Actividad-1-tecnologia-web-2* [Código fuente, revisión ccba795]. GitHub. https://github.com/ROXIAN120/Actividad-1-tecnologia-web-2/tree/ccba795bef45dcad4471650049a2c9aab0a84822/actividad-02

Universidad Privada Domingo Savio. (2026). *Actividad 02 (grupal): De JSON a MySQL* [Consigna y rúbrica del aula virtual, transcritas por el usuario]. https://virtual.upds.edu.bo/mod/assign/view.php?id=1748192

Apéndice A Bitácora consolidada de inteligencia artificial

Responsable: Rodrigo Ledezma Sanchez, grupo HTML. Actualización: 16 de octubre de 2026. Herramienta: Codex (OpenAI, 2026). Se declara asistencia sustancial en código, depuración, pruebas, organización y documentos. No se presenta la solución como autoría humana exclusiva ni se reconstruyen prompts históricos no disponibles.

Etapa	Asistencia y evidencia histórica conservada
Labs 1 y 2	PHP, salida escapada, funciones e inventario. Capturas 12/12 en cada laboratorio.
Labs 3 y 4	Validación, PRG, JSON y operaciones con fotos. Capturas 13/13 en cada uno.
Labs 5 y 6	POO, encapsulamiento, Composer, herencia, enum, trait y contratos. Capturas 14/14.
Lab 7	Router, plantillas y excepciones. Captura 15/15 y error 500 con referencia de log.
Lab 8	PDO y CRUD de productos. Captura 15/15 y búsquedas SQLi antes/después.
Lab 9	Repositorios, páginas, filtros y ficha de socio. Captura 14/14.
Publicación	Carpetas y ramas 1-9 publicadas. Base histórica 399257b.

Los registros particulares siguen en laboratorios/laboratorio-N/BITACORA_IA.md. Los conteos anteriores son resultados históricos documentados, no ejecuciones nuevas en esta entrega. Las capturas históricas proceden del usuario; las capturas nuevas de la aplicación final proceden de navegación automatizada real por el asistente.

Apéndice A (continuación) Intervención final y revisión humana

La revisión del 15 de octubre fue solo documental y dejó pendientes explícitos. El 16 de octubre el usuario solicitó completar todo: “Quiero que completes la actividad 2, con todo lo que pide mas las capturas mas explicacion de codigo, con formato apa 7”. Ese cambio autorizó la implementación actual, distinta de los documentos anteriores de revisión.

Codex generó la aplicación independiente, el esquema normalizado, repositorios, validaciones, importador, protección de formularios y pruebas. Después ejecutó 48 comprobaciones, midió planes EXPLAIN y navegó el CRUD para obtener capturas auténticas. Los registros de demostración se eliminaron y la base volvió a su catálogo inicial.

La revisión corrigió la presentación del modo por clave para que indicara ID ascendente, y distinguió la simulación idempotente de una inserción real de datos ya existentes. También documentó que las medianas de las consultas optimizadas no fueron menores en esta muestra. Estos límites evitan presentar evidencia de rendimiento o migración inexistente.

La IA redactó el informe y el material de defensa con referencias técnicas primarias. No se afirma que el participante haya validado cada afirmación, realizado defensa oral o enviado archivos al aula. Debe revisar contenido, integrantes, fecha y criterios con su docente. La restricción de IA en las guías fue explicada por el usuario como dirigida a sus estudiantes; se conserva esta declaración sin ocultar el uso de asistencia.

El archivo BITACORA_IA.md ofrece la versión de texto del registro, referencias y enlace al repositorio. Este apéndice integra esa declaración en el informe final. La IA no sustituye la responsabilidad por la entrega ni la comprensión del código que se defenderá.

Apéndice B Registro íntegro de las 48 comprobaciones

```
[OK] PDO nativo y base MySQL 8.4
[OK] Tres categorías normalizadas
[OK] Datos iniciales coherentes: 36 productos y 8 socios
[OK] Página 4 contiene seis productos
[OK] Filtro Cera y vela encuentra dos
[OK] Orden precio descendente
[OK] Página grande se ajusta a última
[OK] Inyección SQL en productos no altera consulta
[OK] Comodín porcentaje literal
[OK] Comilla sola se consulta sin error
[OK] Inyección SQL en socios no altera consulta
[OK] Paginación por clave no repite ID
[OK] Crear socio con repositorio
[OK] Actualizar socio y teléfono nullable
[OK] Socio inválido responde 422
[OK] Socios paginados: página 2 con fixtures
[OK] Crear producto: decimal normalizado y relaciones
[OK] Código repetido traduce 1062 a 409
[OK] Actualizar producto solo afecta su código
[OK] Socio asociado: eliminación responde 409
[OK] Producto inválido responde 422
[OK] PDO FK impide relación inexistente
[OK] PDO CHECK impide precio negativo
[OK] Migración JSON: simulación revierte cambios
[OK] Migración JSON: inserción confirmada
```

Apéndice B (continuación) Registro íntegro de las 48 comprobaciones

```
[OK] Migración JSON idempotente sin sobrescribir
[OK] Token CSRF generado
[OK] POST sin token rechazado 403
[OK] GET no elimina y responde 405
[OK] Ruta inexistente responde 404
[OK] Página no numérica vuelve a 1
[OK] Orden SQL desconocido no ejecuta instrucciones
[OK] XSS reflejado escapado en búsqueda
[OK] Alta HTTP socio usa PRG 303
[OK] XSS almacenado se muestra como texto
[OK] Editar socio HTTP responde 303
[OK] Validación HTTP socio conserva entrada escapada
[OK] Validación HTTP producto responde 422
[OK] Crear producto HTTP 303
[OK] Editar producto HTTP 303
[OK] Eliminar producto HTTP 303
[OK] Eliminar socio HTTP 303
[OK] Cabeceras CSP y nosniff
[OK] Configuración privada no accesible por HTTP
[OK] Clave ID disponible por HTTP
[OK] Eliminar producto en repositorio
[OK] Eliminar inexistente responde 404
[OK] Fixtures limpiados sin restaurar tablas
Resultado: 48 de 48 comprobaciones
```

Este registro corresponde a la ejecución final verificada antes de empaquetar. El JSON acompañante identifica fecha y entorno. La limpieza selectiva se comprueba al final; no se usó una restauración global para simular que las operaciones funcionaron.

Apéndice C Extracto del script SQL implementado

```
-- Inicialización exclusiva de la base nueva actividad02. No contiene DROP.
SET NAMES utf8mb4;
CREATE TABLE categorias (
 id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
 nombre VARCHAR(60) NOT NULL,
 descripcion VARCHAR(150) NOT NULL,
 CONSTRAINT uq_categoria_nombre UNIQUE(nombre)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
CREATE TABLE socios (
 id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
 nombre VARCHAR(80) NOT NULL,
 comunidad VARCHAR(60) NOT NULL,
 telefono VARCHAR(20) NULL,
 CONSTRAINT chk_socio_nombre CHECK(CHAR_LENGTH(TRIM(nombre)) BETWEEN 3 AND 80),
 CONSTRAINT chk_socio_comunidad CHECK(CHAR_LENGTH(TRIM(comunidad)) BETWEEN 2 AND 60),
 INDEX idx_socio_nombre(nombre,id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
CREATE TABLE productos (
 id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
 codigo VARCHAR(8) NOT NULL,
 nombre VARCHAR(80) NOT NULL,
 categoria_id INT UNSIGNED NOT NULL,
 precio DECIMAL(10,2) NOT NULL,
 stock INT NOT NULL DEFAULT 0,
 socio_id INT UNSIGNED NOT NULL,
 CONSTRAINT uq_producto_codigo UNIQUE(codigo),
 CONSTRAINT fk_producto_categoria FOREIGN KEY(categoria_id) REFERENCES categorias(id) ON
     DELETE RESTRICT,
 CONSTRAINT fk_producto_socio FOREIGN KEY(socio_id) REFERENCES socios(id) ON DELETE
    RESTRICT,
 CONSTRAINT chk_producto_precio CHECK(precio>0 AND precio<=10000),
 CONSTRAINT chk_producto_stock CHECK(stock BETWEEN 0 AND 100000),
 CONSTRAINT chk_producto_nombre CHECK(CHAR_LENGTH(TRIM(nombre)) BETWEEN 3 AND 80),
 INDEX idx_producto_nombre(nombre,id),
 INDEX idx_producto_categoria_nombre(categoria_id,nombre,id),
 INDEX idx_producto_socio_nombre(socio_id,nombre,id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

El script íntegro y la semilla se incluyen como archivos .sql ejecutables. Este extracto reproduce el esquema, no una propuesta pendiente. Las consultas de la aplicación utilizan parámetros; los scripts de inicialización contienen únicamente datos de ejemplo constantes.

Apéndice D Guion de defensa y preguntas

Duración orientativa: siete a diez minutos. Presentar primero el problema del catálogo JSON y las tres tablas. Mostrar las FK y explicar por qué un socio con productos no se elimina. Seguir un alta desde el formulario al validador, repositorio y sentencia preparada.

Demostrar CRUD con registros temporales y luego borrarlos en orden: producto antes que socio. Mostrar un formulario inválido, SQLi sin coincidencias y XSS como texto. Ejecutar las 48 pruebas en otra terminal y abrir los planes medidos. No exponer .env en pantalla.

Preguntas de preparación: ¿Por qué DECIMAL y no float para persistir dinero? ¿Qué diferencia hay entre validar en PHP y aplicar CHECK en SQL? ¿Por qué ORDER BY necesita una lista permitida? ¿En qué cambia OFFSET frente a clave ID? ¿Por qué leer menos filas no prueba siempre menor tiempo? ¿Qué hicieron el participante y la IA?

Responder con los archivos entregados, no con afirmaciones generales. La solución no incluye autenticación ni roles y está limitada al entorno local. La presentación acompaña esta defensa, pero el PDF no acredita que la exposición ya haya ocurrido. Consultar GUIA_DEFENSA.md para comandos, rutas reproducibles y respuestas desarrolladas.